

Why the Repository Pattern Exists (in Plain English)

Imagine your app without a repository

Say your app has 5 different places that need students:

1. Program.cs — adds a new student when the app starts
2. A “Save Student” button in a form
3. A “Search Students” feature
4. A “Delete Student” feature
5. A report that lists all students

Without a repository, each of these 5 places would write EF Core code directly, like this:

```
// in Program.cs
_context.Students.Add(student);
_context.SaveChanges();

// in the Save button
_context.Students.Add(student);
_context.SaveChanges();

// in Search Students
_context.Students.Where(s => s.StudentName == name).ToList();

// in Delete Students
_context.Students.Remove(student);
_context.SaveChanges();

// in the report
_context.Students.ToList();
```

Same kind of code, copy-pasted in 5 different files.

Problem 1: “Switch database tools”

Let’s say one year from now, your company decides: “We’re not using EF Core anymore, we’re switching to Dapper” (a different tool for talking to databases).

Now you have to go into **all 5 places**, find every line that uses `_context.Students`, and rewrite it in Dapper’s syntax instead.

- If you had 5 places, that’s annoying but doable.
- If you had 50 places across a big real app, that’s a nightmare — you might miss one and break something.

“Hunt down and rewrite EF Core code scattered across your entire project” literally means: search your whole codebase for every spot where you used `_context.Students...`, and change each one, one by one.

Problem 2: “Test your code without a real database”

Imagine you want to test: “does my Delete feature actually mark a student for deletion correctly?”

Normally, to test that, you’d need:

- A real SQL Server running
- A real database with real data in it
- Your test would actually delete from that real database

That’s slow, and risky (you might accidentally test on real data).

What programmers prefer is to test using a **fake, pretend database** that lives only in memory, so tests run fast and don’t touch anything real.

But if your `Program.cs`, your buttons, and your reports all talk **directly** to `_context.Students` (the real EF Core connection), there’s no easy way to swap in a “fake” one just for testing — because that real connection is hardwired everywhere.

The Repository’s Fix, in Plain Words

Instead of every part of your app saying “talk to EF Core directly,” every part instead says “talk to the repository.” The repository is the **only** place that actually knows about EF Core.

So now:

- If you switch to Dapper → you only rewrite the code **inside one file** (`GenericRepo.cs`), not five or fifty.
- If you want a fake database for testing → you can create a second, fake version of the repository (following the same `IGenericRepo` promise/contract) and swap it in for testing, without touching `Program.cs` or anything else.